

Tecan Freedom EVO

电机与运动控制策略

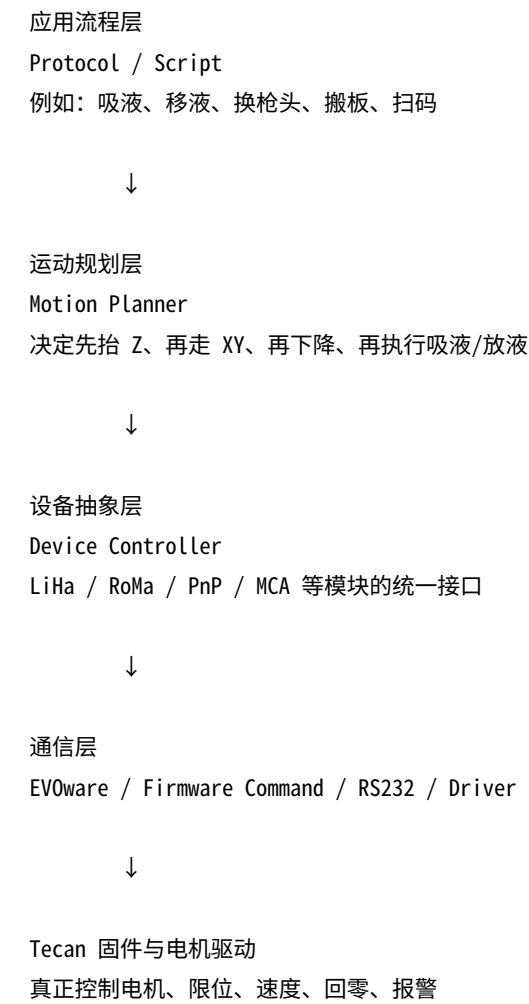
整理版技术说明文档

适用于 CSU-ZhongRui-Tecan-Freedom-EVO 项目的上层运动控制设计。

核心观点：Freedom EVO 的底层电机控制、限位、回零和硬件状态反馈通常由 Tecan 固件与 EVOware 驱动层封装；项目代码更适合实现上层运动规划、安全路径、状态机、命令队列和异常处理。

1. 总体控制层级

Tecan Freedom EVO 的“电机控制策略”不建议理解成自己直接写 PID 去控制每个电机。更合理的做法是把底层电机控制视为设备固件能力，把项目重点放在上层运动控制策略。



2. Freedom EVO 上主要运动对象

模块	作用	控制重点
LiHa	Liquid Handling Arm，移液臂	X/Y/Z 位置、吸液高度、分液高度、Tip 间距
RoMa	Robotic Manipulator Arm，机械搬运臂	搬板、抓取、旋转、放置
PnP	Pick and Place	单管/小物件搬运
MCA 96/384	多通道移液臂	整板吸液/分液、Z 轴安全高度
Diluter/Syringe	注射泵/稀释器	体积、流速、液面检测相关逻辑

3. 推荐的电机/运动控制策略

A. 初始化策略：先回零，再进入工作状态

Power On

Tecan Freedom EVO 电机与运动控制策略 | 整理版

```

↓
Check Connection
↓
Initialize Arms
↓
Homing / Reference Position
↓
Check Deck Layout
↓
Ready

```

项目里可以把设备行为管理成明确状态，避免在未知状态下继续运动。

```

class MotionState:
    IDLE = "idle"
    INITIALIZING = "initializing"
    READY = "ready"
    MOVING = "moving"
    ERROR = "error"
    EMERGENCY_STOP = "emergency_stop"

```

B. 安全路径策略：Z 轴优先避障

对于实验室自动化设备，最重要的不是最快移动，而是不撞枪头、不撞板、不撞载架、不洒液。推荐路径如下：

```

当前点
↓
Z 轴抬升到安全高度
↓
XY 平移目标上方
↓
Z 轴下降到操作高度
↓
执行吸液 / 分液 / 抓取 / 放置
↓
Z 轴再次抬升

```

```

def safe_move_to(controller, x, y, z, safe_z):
    controller.move_z(safe_z)
    controller.move_xy(x, y)
    controller.move_z(z)

```

C. 坐标管理策略：不要直接写死电机步数

不要在代码里直接写电机步数，而应该建立 deck 坐标模型，让上层代码调用位置名称。

```

DECK_POSITIONS = {
    "plate_1_A1": {"x": 120.5, "y": 80.2, "z": 35.0},
    "tip_rack_1_A1": {"x": 40.0, "y": 50.0, "z": 28.5},

```

```

    "waste": {"x": 300.0, "y": 40.0, "z": 60.0},
}

move_to_position("plate_1_A1")

```

4. 控制策略核心：状态机 + 命令队列

```

IDLE
  ↓ initialize
READY
  ↓ move command
MOVING
  ↓ success
READY
  ↓ error
ERROR
  ↓ reset
READY

class MotionCommand:
    def __init__(self, axis, target, speed=None):
        self.axis = axis
        self.target = target
        self.speed = speed

class MotionQueue:
    def __init__(self):
        self.commands = []

    def add(self, command):
        self.commands.append(command)

    def run(self, controller):
        for command in self.commands:
            controller.execute(command)
            controller.wait_until_done()
            controller.check_error()

```

5. 对 LiHa 的控制策略

```

取枪头
  ↓
移动到液体上方
  ↓
下降到吸液高度
  ↓

```

吸液
 ↓
 抬升
 ↓
 移动到目标孔位
 ↓
 下降到分液高度
 ↓
 分液
 ↓
 抬升
 ↓
 丢枪头

控制点	策略
Z 轴高度	每种 labware 单独配置吸液/分液高度
速度	吸液、分液、移动速度分开配置
液面检测	有条件时优先使用 liquid level detection
防撞	所有 XY 移动前先抬 Z
防污染	移液后可加 blowout、touch tip、air gap
异常	吸液失败、Tip 未取到、液面未检测到时进入 ERROR

6. 对 RoMa 的控制策略

移动到源板位上方
 ↓
 下降
 ↓
 夹爪闭合
 ↓
 确认抓取
 ↓
 抬升到安全高度
 ↓
 移动到目标板位
 ↓
 下降
 ↓
 夹爪打开
 ↓
 抬升
 永远不要斜着低空穿越 deck
 抓板后先升到 transport height
 放板前确认目标位置为空
 移动 vector 需要人工校验
 异常时保持夹爪状态，不要盲目释放

7. 速度控制策略

公开资料通常不会给出 Tecan 底层电机 PID 参数，所以项目里不应该假设自己能直接调 PID。建议上层只定义运动速度等级。

```
class SpeedProfile:
    SLOW = "slow"      # 精细定位、接近液面、抓取/放置
    NORMAL = "normal"  # 普通移动
    FAST = "fast"      # 空载、安全高度移动
```

场景	速度
空载 XY 移动	Fast
枪头接近液面	Slow
RoMa 抓板下降	Slow
RoMa 搬运途中	Normal/Fast
放板前下降	Slow
异常恢复	Slow

8. 异常处理策略

```
运动超时
限位触发
通信中断
初始化失败
液面检测失败
Tip 未取到
RoMa 抓取失败
碰撞风险
用户急停
def handle_motion_error(error):
    stop_all_motion()
    log_error(error)
    set_state("ERROR")
    wait_for_operator()
```

不要自动乱动恢复，尤其是 RoMa 抓着板子或 LiHa 带着枪头时。更安全的方式是：暂停、停止动作、记录错误、等待人工确认，然后低速恢复或重新初始化。

9. 项目代码建议结构

```
CSU-ZhongRui-Tecan-Freedom-EVO/
|
|—— main.py
|—— README.md
|
|—— motion_control/
|    |—— __init__.py
```

```

|   |—— controller.py      # 总控制器
|   |—— state_machine.py   # 状态机
|   |—— command_queue.py   # 命令队列
|   |—— safety.py          # 避障、安全高度、急停
|   |—— deck_map.py        # 工作台坐标
|   |—— liha.py            # 移液臂控制
|   |—— roma.py           # 机械臂控制
|   |—— config.py         # 参数配置
|
|—— protocols/
|   |—— sample_transfer.py
|   |—— plate_move.py
|
|—— tests/
|   |—— test_motion_queue.py
|   |—— test_safety.py

```

10. 可直接写进 README 的版本

电机与运动控制策略

本项目不直接修改 Tecan Freedom EVO 的底层电机驱动参数，而是在上层实现运动控制逻辑。底层电机控制、限位检测、回零和硬件状态反馈由 Freedom EVO 固件及 EVOware 驱动层完成。

本项目的运动控制策略包括：

1. 初始化与回零：运行前对 LiHa、RoMa 等机械臂进行初始化；
2. 安全路径规划：所有 XY 移动前先提升 Z 轴到安全高度；
3. 坐标抽象：通过 deck map 管理载架、孔位、枪头盒和废液槽位置；
4. 状态机控制：使用 IDLE、INITIALIZING、READY、MOVING、ERROR 等状态管理设备行为；
5. 命令队列：将复杂流程拆解为 MoveZ、MoveXY、Aspirate、Dispense、Grip、Release 等命令；
6. 异常保护：对超时、碰撞风险、液面检测失败、抓取失败等情况进行停机和人工确认；
7. 仿真验证：在真实运行前使用 EVOware/EVOSim 或模拟接口验证运动路径。

11. 一句话总结

Freedom EVO 的电机控制策略应该做成“上层运动规划 + 安全路径 + 状态机 + 命令队列”，不要试图在项目里直接重写底层电机闭环控制。

参考来源

- Tecan Freedom EVOware Getting Started Guide, version 2.7. 用于理解 EVOware 的脚本、工作台定义、EVOSim、机械臂初始化与运动向量相关概念。
- Tecan Freedom EVO 75 brochure/specification. 用于理解设备定位重复性等上层运动规划相关指标。